ESCOLA
SUPERIOR
DE TECNOLOGIA
E GESTÃO

P.PORTO

Sistemas Operativos

Licenciatura em Engenharia Informática

Licenciatura em Segurança Informática em Redes de Computadores

Sistemas Operativos

Processos e Threads

Agenda

- Definições: Processo e Threads
- Comparação e diferenças
- Quando usar Processo vs Threads
- Criando Processos em Java
- Criando Threads em Java
- Conclusão

Definições

O que são processos?

- Um **processo** é uma instância de um programa em execução.
- Ele tem seu próprio espaço de memória, variáveis, recursos e geralmente é isolado de outros processos.
- Quando você cria um processo em Java, você está iniciando um programa externo ao seu próprio código Java.

O que está associado a um processo?

- Associado a cada processo está o **espaço de endereçamento (posições de memória):** contém o programa executável, os dados do programa e a sua pilha.
- Além do espaço de endereçamento: conjunto de registradores (pc, sp), lista de arquivos abertos, etc...
- Um processo é na essência um **contêiner** que armazena todas as informações necessárias para executar um programa.

Como são criados?

- Processos são criados em Java através das classes **Process** e **ProcessBuilder**.
- Isso faz com que o sistema operativo inicie um novo programa fora do seu programa Java.
- Processos são usados para rodar outros programas no sistema operativo, como comandos de terminal, scripts, ou aplicativos externos.

O que são Threads?

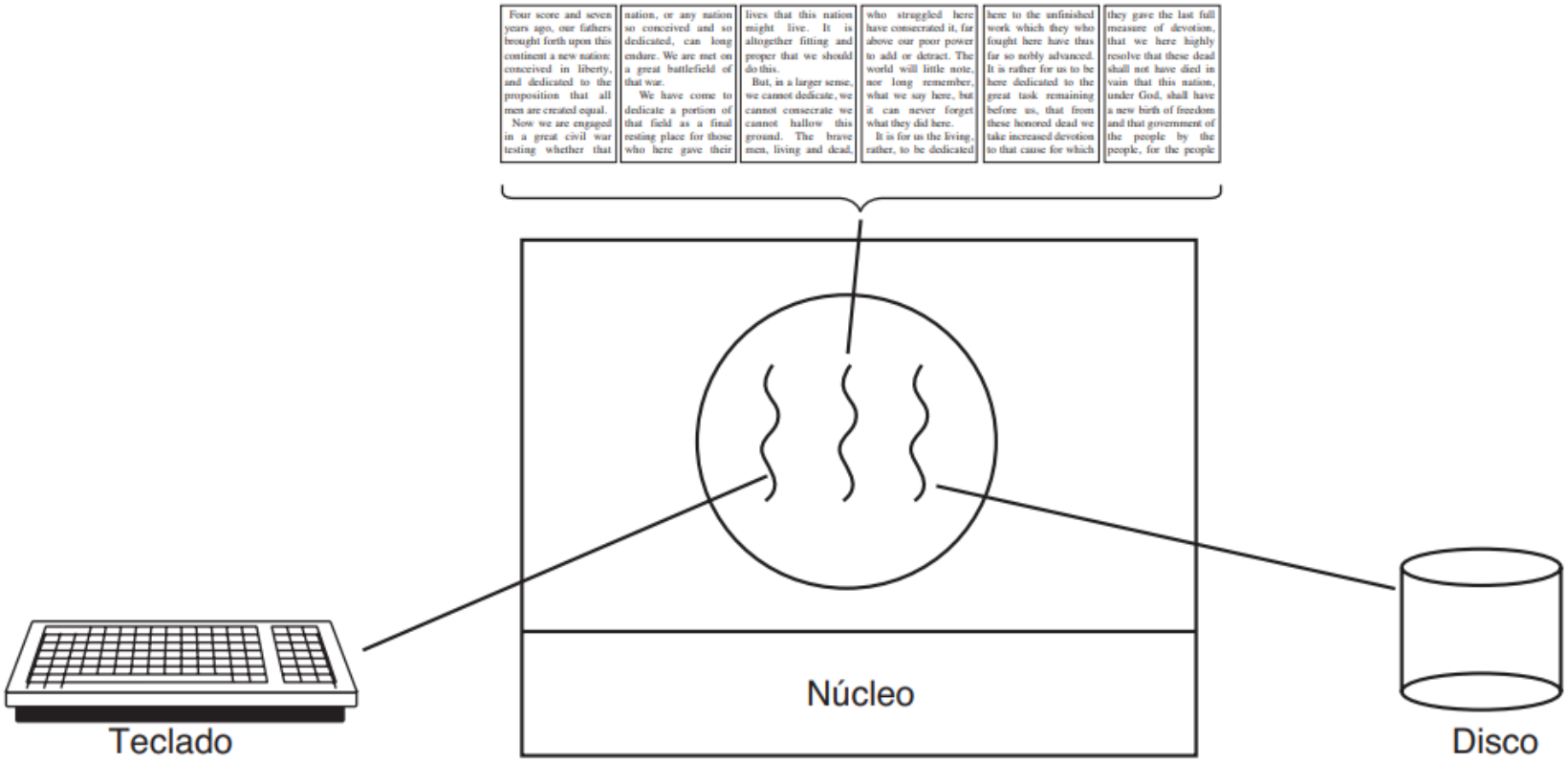
linhas, fios, tópicos

- Uma **thread** é uma unidade de execução dentro de um processo.
- Diferente dos processos, threads compartilham o mesmo espaço de memória dentro de um processo, permitindo que diferentes partes do código rodem simultaneamente dentro do mesmo programa.
- OBS: threads são particularmente úteis em sistemas com múltiplas CPUs, onde o paralelismo real é possível.

Exemplo de uso de Threads

- Usado para **rodar tarefas em paralelo** dentro do **mesmo programa**, como realizar operações simultâneas.
- Por exemplo, um download em segundo plano enquanto a interface do programa continua responsiva.

Processador de texto com 3 threads



Comparação: Processos x Threads

- **Isolamento:**
  - Processos são completamente isolados uns dos outros.
  - Threads compartilham o mesmo espaço de memória.
- **Memória:**
  - Cada processo tem seu próprio espaço de memória.
  - Todas as threads de um processo compartilham a memória.

Comparação: Processos x Threads

- **Criação:**
  - Processos são criados com *ProcessBuilder* ou *Runtime.exec()*
  - Threads são criadas com *Thread* ou *Runnable*.

- **Overhead:**
  - Processo é mais pesado: iniciar um novo processo requer mais recursos do sistema.
  - Thread é mais leve: threads têm menos sobrecarga do que processos.

Comparação: Processos x Threads

- **Execução concorrente:**
  - Processos são executados de forma independente.
  - Threads são executadas em paralelo dentro do mesmo processo.

- **Finalidade:**
  - Processos: executar programas externos ou novos processos do sistema operacional.
  - Thread: Executar múltiplas tarefas simultâneas dentro do mesmo programa.

Como criar um processo em Java?

- **Class Runtime:**
  - Permite interagir com o ambiente no qual o programa Java está sendo executado. Através dela, podemos executar comandos do sistema operacional.
  - Método principal: Runtime.getRuntime().exec()

```
Runtime runtime = Runtime.getRuntime();

Process process = runtime.exec("ls -l");
```

Module java.base

Package java.lang

Class Runtime

java.lang.Object
    java.lang.Runtime

```
public class Runtime
extends Object
```

Every Java application has a single instance of class `Runtime` that allows the application to interface with the environment in which the application is running. The current runtime can be obtained from the `getRuntime` method.

An application cannot create its own instance of this class.

- https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Runtime.html

Como criar um processo em Java?

- **Class ProcessBuilder:**
  - Foi introduzida para substituir o uso direto da **Runtime**, oferecendo mais controlo e flexibilidade ao criar e gerenciar processos. Ela permite modificar o ambiente do processo, redirecionar sua entrada/saída, e mais.
  - Métodos principais:

    start()

    redirectInput()

    redirectOutput()

    environment()

Module java.base

Package java.lang

Class ProcessBuilder

java.lang.Object
    java.lang.ProcessBuilder

```
public final class ProcessBuilder
extends Object
```

This class is used to create operating system processes.

- https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/ProcessBuilder.html

ProcessBuilder

- **Objeto ProcessBuilder:**
  - O primeiro passo é criar um objeto ProcessBuilder e passar o comando a ser executado como argumento.

```
ProcessBuilder pb = new ProcessBuilder("comando");

ProcessBuilder pb = new ProcessBuilder(args[0]);


ProcessBuilder pb = new ProcessBuilder("comando", "arg1", "arg2");

ProcessBuilder pb = new ProcessBuilder(args);
```

ProcessBuilder

- **Iniciar o processo:**
  - Depois de configurar o ProcessBuilder, o processo é iniciado com o método start(), que retorna um objeto do tipo Process.

```
Process process = pb.start();

ou diretamente...

Process process = new ProcessBuilder("comando").start();
```

ProcessBuilder

- **Configurar o processo:**
  - Antes de iniciar o processo é possível também, opcionalmente, configurar alguns aspetos como a E/S.

```
pb.redirectOutput(new File("saida.txt"));


pb.redirectInput(new File("entrada.txt"));
```

Classe Process

- A classe **Process** em Java representa um processo do sistema operativo que foi criado e está a ser executado a partir de um programa Java.

- Ela fornece métodos para interagir com o processo enquanto ele está em execução ou depois que ele termina.

https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Process.html

Module java.base

Package java.lang

Class Process

java.lang.Object
    java.lang.Process

```
public abstract class Process
extends Object
```

`Process` provides control of native processes started by ProcessBuilder.start and Runtime.exec. The class provides methods for performing input from the process, performing output to the process, waiting for the process to complete, checking the exit status of the process, and destroying (killing) the process. The `ProcessBuilder.start()` and `Runtime.exec` methods create a native process and return an instance of a subclass of `Process` that can be used to control the process and obtain information about it.

Ler a saída (output) do processo

- O método **getInputStream()** do objeto **Process** retorna um fluxo de dados com a **saída padrão do processo**.

```
InputStream is = process.getInputStream();
```

- A classe **InputStreamReader** lê bytes de um fluxo de entrada (**InputStream**) e os converte em caracteres usando uma codificação de caracteres especificada (como UTF-8, ISO-8859-1, etc.).

```
InputStreamReader isr = new InputStreamReader(is);
```

Ler a saída (output) do processo

- A classe **BufferedReader** é usada para ler texto de um fluxo de entrada de maneira eficiente.
- Fornece a capacidade de ler grandes blocos de dados ao invés de um único caractere por vez.
- Ela é comumente usada em conjunto com classes como **FileReader**, **InputStreamReader**.

```
BufferedReader br = new BufferedReader(isr);
```

Ler a saída (output) do processo

- Finalmente...

```
InputStream is = process.getInputStream();
InputStreamReader isr = new InputStreamReader(is);
BufferedReader br = new BufferedReader(isr);
// ler o output do processo
String line;
while((line = br.readLine()) != null)
    System.out.println(line);
br.close();
```

Ler a saída (output) do processo

Module java.base
Package java.io

Class InputStream

java.lang.Object
    java.io.InputStream

https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/io/InputStream.html

Module java.base
Package java.io

Class InputStreamReader

java.lang.Object
    java.io.Reader
        java.io.InputStreamReader

https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/io/InputStreamReader.html

Module java.base
Package java.io

Class BufferedReader

java.lang.Object
    java.io.Reader
        java.io.BufferedReader

https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/io/BufferedReader.html

Como criar uma thread em Java?

- Estendendo a classe Threads
- Implementando a interface Runnable
- Expressões lambda (Java 8 ou superior)

Como criar uma thread em Java?

- Estendendo a classe Threads
- **Implementando a interface Runnable**
- **Expressões lambda (Java 8 ou superior)**

Estendendo a classe Threads

```java
class OlaMundoThread extends Thread {

    @Override
    public void run() {

    // Código que será executado pela thread
        System.out.println(x:"Olá Mundo!");

    }
}

public class ExemploThread {
    Run | Debug
    public static void main(String[] args) {
        OlaMundoThread thread = new OlaMundoThread(); // Instancia uma nova thread
        thread.start(); // Inicia a execução da thread
    }
}
```

Implementando a interface *Runnable*

```java
class OlaMundoThread implements Runnable {

    @Override
    public void run() {

    // Código que será executado pela thread
        System.out.println("Olá Mundo!");

    }
}

public class ExemploRunnable {
    Run | Debug
    public static void main(String[] args) {
        OlaMundoThread tarefa = new OlaMundoThread(); // Instancia o objeto Runnable
        Thread thread = new Thread(tarefa); // Passa o Runnable para a thread
        thread.start(); // Inicia a execução da thread
    }
}
```

Module java.base
Package java.lang

Interface Runnable

All Known Subinterfaces:

`RunnableFuture<V>`, `RunnableScheduledFuture<V>`

All Known Implementing Classes:

`AsyncBoxView.ChildState`, `ForkJoinWorkerThread`, `FutureTask`, `RenderableImageProducer`, `SwingWorker`, `Thread`, `TimerTask`

Functional Interface:

This is a functional interface and can therefore be used as the assignment target for a lambda expression or method reference.

```
@FunctionalInterface
public interface Runnable
```

The `Runnable` interface should be implemented by any class whose instances are intended to be executed by a thread. The class must define a method of no arguments called `run`.

https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Runnable.html

Expressões lambda

```java
public class ExemploLambda {
    Run | Debug
    public static void main(String[] args) {
        Thread tarefa = new Thread(() -> {
            System.out.println(x: "Olá Mundo!");
        });
        tarefa.start();
    }
}
```

```java
public class ExemploLambda {
    public static void main(String[] args) {
        new Thread(() -> {
            System.out.println("Olá Mundo!");
        }).start();
    }
}
```

Conclusão

- **Processos** são unidades independentes de execução que rodam fora do seu programa Java e possuem seu próprio espaço de memória. Use processos para executar programas externos.
- **Threads** são unidades de execução concorrentes dentro do mesmo programa, compartilhando o mesmo espaço de memória. Use threads para executar tarefas paralelas dentro do seu programa.

ESCOLA
SUPERIOR
DE TECNOLOGIA
E GESTÃO

P.PORTO

Sistemas Operativos

Licenciatura em Engenharia Informática

Licenciatura em Segurança Informática em Redes de Computadores